Lecture 05.08: Class Methods (@classmethod)

📌 Executive Summary

The @classmethod decorator, cls parameter binding, modifying class-level state, and factory constructor design patterns.

🔬 Theoretical Foundations & Principles

1. The @classmethod Decorator

A class method is bound to the class itself rather than to an instance of the class:

```
class MyClass:
    @classmethod
    def my_method(cls, arg1):
        ...
```

When invoked as `MyClass.my_method()` or `instance.my_method()`, CPython automatically passes the class object as the first argument (`cls`).

2. Polymorphic Factory Pattern

Class methods are ideal for alternative constructors because `cls` dynamically references the concrete derived class upon inheritance.

💻 Production Implementation & Code

```
class User:
    total_users = 0

    def __init__(self, username: str, role: str):
        self.username = username
        self.role = role
        User.total_users += 1

    @classmethod
    def create_admin(cls, username: str):
        """Factory method to create an admin user."""
```

```
        return cls(username, role="ADMINISTRATOR")

    @classmethod
    def get_user_count(cls) -> int:
        return cls.total_users

u1 = User("Alice", "MEMBER")
u2 = User.create_admin("Bob")
print(f"Total Users: {User.get_user_count()} | Bob's Role: {u2.role}")
```

⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Method dispatch overhead is $\mathcal{O}(1)$ via CPython descriptor resolution.
- **Auxiliary Space:** Instance attribute memory allocation in `__dict__` vs `__slots__`.